

```
+-----+-----+-----+-----+
| RESULTADO DEL BLOQUE 2 |
+-----+-----+-----+-----+
| expresion | predijiste | salio | |
+-----+-----+-----+-----+
| 7 / 2 | 3 | 3 | ok |
| 7.0 / 2 | 3.5 | 3.5 | ok |
| (double) 7 / 2 | 3.5 | 3.5 | ok |
| (double) (7 / 2) | 3 | 3.0 | ok |
| 7.0 / 0 | infinity | Infinity | ok |
| 0.0 / 0.0 | null | NaN | FALLO |
+-----+-----+-----+-----+

Fallaste 1. Anotalas: explicarlas es lo que se califica.
```

Act 3: En todas las operaciones de este bloque predije valores lógicos (true o false). Me confundí pensando que el símbolo % evaluaba una condición, pero en realidad es el operador matemático de módulo, el cual devuelve el residuo numérico de una división. Por eso $7 \% 2$ arrojó 1, y $10 \% 5$ arrojó 0

expresion	predijiste	salio	
$7 \% 2$	false	1	FALLO
$10 \% 5$	true	0	FALLO
$-7 \% 2$	false	-1	FALLO
$7 \% 2.5$	false	2.0	FALLO
$12345 \% 10$	true	5	FALLO
$12345 / 10$	1234	1234	ok

Act 4: Predije que el sistema arrojaría los números 6 y 65. Mi error fue no considerar que el casting explícito a (char) obliga a Java a traducir el valor numérico a su símbolo correspondiente en la tabla ASCII; por lo tanto, el 65 se convierte en la letra A, y la suma de 'A' + 1 (que es 66) da como resultado B.

```
(char) ('A' + 1) = 65
(byte) 200 = -56
```

RESULTADO DEL BLOQUE 4			
expresion	predijiste	salio	
(int) 3.9	3	3	ok
(int) -3.9	-3	-3	ok
(char) 65	6	A	FALLO
(int) 'A'	65	65	ok
'A' + 1	66	66	ok
(char) ('A' + 1)	65	B	FALLO
(byte) 200	-56	-56	ok

Fallaste 2. Anotalas: explicarlas es lo que se califica.

Act 5: Predije que la igualdad sería true y que la suma daría 0.3 exacto. El programa arrojó false y un resultado de 0.300000~. Esto ocurre por un problema clásico de precisión de punto flotante: la arquitectura de la computadora no puede representar decimales con precisión infinita en binario, dejando minúsculos residuos en la memoria.

RESULTADO DEL BLOQUE 5			
expresion	predijiste	salio	
5 > 3	true	true	ok
5 == 5.0	true	true	ok
0.1 + 0.2 == 0.3	true	false	FALLO
true && false	false	false	ok
true false	true	true	ok
vidasRestantes > 0 && 10 / vi~	false	false	ok

Fallaste 1. Anotalas: explicarlas es lo que se califica.

Act 6: Pensé que el resultado sería la suma 15. Fallé porque Java evalúa la expresión de izquierda a derecha; como el primer elemento ("1") es un String, el compilador asume que toda la operación es una concatenación de texto, simplemente pegando los caracteres para formar "123" en lugar de sumar sus valores matemáticos.

RESULTADO DEL BLOQUE 6			
expresion	predijiste	salio	
0.1 + 0.2	0.3	0.300000~	FALLO
Integer.MAX_VALUE + 1	-2147483648	-2147483~	ok
5 + 3	8	8	ok
"5" + 3	53	53	ok
1 + 2 + "3"	33	33	ok
"1" + 2 + 3	15	123	FALLO